Pioneer Delivery Company's Algorithm Portfolio

Part III: Dynamic Programming

Project Overview

Pioneer Delivery is a growing local delivery company that receives many delivery requests each day. Each request occupies a driver for a specified time interval. In **Part I**, you used greedy algorithms to schedule deliveries and determine the number of drivers needed. In **Part II**, you practiced divide-and-conquer techniques to solve additional problems efficiently.

Pioneer is now expanding its planning software. Some delivery requests earn more revenue than others, the company must manage limited resources, and its digital event logs need to retain critical information. You will implement and analyze dynamic programming solutions for the following tasks:

- **Task A:** Select a compatible set of deliveries with *maximum total profit* (weighted interval scheduling).
- **Task B:** Select deliveries that maximize profit under a *limited fuel budget* (0/1 knapsack).
- **Task C:** Remove unnecessary event-log entries while preserving a required *ordered sequence of events* (sequence alignment / subsequence DP; LeetCode #3316, Medium).
- **Task D:** Prepare a *README*, algorithm documentation, test cases, and a reproducible submission.

Total: 100 points. You may use a programming language of your choice.

Task A Maximum-Profit Delivery Scheduling (35 points)

Suppose Pioneer has **one driver** available. Each delivery request has four fields:

$$(\text{id},\ \text{start},\ \text{finish},\ \text{profit}).$$

The driver can complete only one delivery at a time. A delivery finishing at time t is compatible with another delivery starting at time t. You may assume start $<$ finish and profit ≥ 0 for every request.

Previously, a greedy earliest-finish-time algorithm was sufficient to *maximize the number* of deliveries. It does *not* necessarily maximize total profit. For this task, you must use **weighted interval scheduling with dynamic programming**.

Example

Delivery ID	Start	Finish	Profit
A	0	3	50
B	1	4	60
C	3	5	40
D	4	7	70
E	5	8	50
F	7	9	50

A maximum-profit compatible schedule is **B, D, F**, with total profit $60 + 70 + 50 = \boxed{180}$. By contrast, the earliest-finish-time schedule **A, C, E** earns only 140.

Requirements

A1. Preprocessing (8 points). Sort the requests by nondecreasing finish time. For each sorted request i, compute $p(i)$, the index of the last request that finishes no later than the start of request i; use $p(i) = 0$ if none exists. Use binary search so that all predecessor indices can be found in $O(n \log n)$ total time.

A2. Dynamic programming (12 points). Implement an algorithm to compute the maximum obtainable total profit. Define your DP state, base case, and recurrence in your report. Do not use exhaustive search or a greedy algorithm as the main solution.

A3. Recover the schedule (8 points). Return both the maximum profit and the IDs of the selected deliveries in chronological order. The returned requests must not overlap. If several optimal schedules exist, returning any one is acceptable.

A4. Analysis (7 points). Explain why the greedy method from Part I can fail when profits differ. Give the time and space complexity of your implementation, including sorting, predecessor computation, and schedule reconstruction.

Expected performance: $O(n \log n)$ time, with $O(n)$ auxiliary space for the DP and predecessor arrays (in addition to any storage needed by your sorting implementation).

Task B Delivery Selection with a Fuel Budget (25 points)

Pioneer has another group of potential deliveries to distribute among **multiple available drivers**. For this task, assume scheduling conflicts are not an issue: any chosen combination can be completed. However, the company has a limited fuel budget for this group of deliveries.

Each candidate delivery has a **fuel cost** (a positive integer number of fuel units) and a **profit** (a nonnegative integer). Each delivery may be accepted *at most once*. Given a total fuel budget B, select a subset with total fuel cost at most B and maximum total profit.

This is the **0/1 knapsack problem**. A delivery cannot be partially accepted.

Example

Delivery ID	Fuel cost	Profit
P	4	20
Q	3	14
R	5	29
S	2	10
T	6	31

For a fuel budget of $B = 10$ units, one optimal selection is **Q, R, S**, with total fuel cost $3 + 5 + 2 = 10$ and maximum profit $14 + 29 + 10 = \boxed{53}$.

Requirements

B1. DP formulation (5 points). Define a state representing the best profit obtainable from the first i deliveries given a fuel budget of b units. State its base cases and recurrence.

B2. Implementation (12 points). Implement a bottom-up dynamic programming solution using a two-dimensional DP table. Your algorithm should return the maximum possible total profit. It must handle an empty input and a budget of zero.

B3. Recover the selected deliveries (5 points). Return the selected IDs together with their total fuel cost and total profit. Verify that no delivery is selected twice and the budget is respected.

B4. Analysis (3 points). Give the time and space complexity in terms of n and B. Explain briefly why this running time depends on the *numerical value* of the budget.

Expected performance: $O(nB)$ time and $O(nB)$ space, using a two-dimensional DP table to support reconstruction.

Task C Event Log Cleanup with Sequence Alignment (20 points)

Technical interview connection: This task is adapted from LeetCode #3316: Find Maximum Removals From Source String (Medium), a problem that extends subsequence matching with dynamic programming.

Pioneer records dispatch events as a string `source`, with each character denoting an event type. A shorter string `pattern` specifies essential events that must remain a **subsequence**: its characters appear in order, but need not be consecutive.

To reduce storage, delete as many characters as possible, choosing only from the zero-based original positions in `targetIndices`, while keeping `pattern` as a subsequence. Deleted positions do *not* cause other indices to shift.

Assume the strings are nonempty and lowercase, `pattern` is initially a subsequence of `source`, and `targetIndices` is a sorted array of distinct valid indices.

Example 1

```
Input:
source = "abbaa"
pattern = "aba"
targetIndices = [0, 1, 2]

Output: 1
```

Explanation:

We cannot remove `source[0]` because doing so would make it impossible to preserve `pattern` as a subsequence. However, we can perform either of the following operations:

- Remove `source[1]`, so that `source` becomes `"a_baa"`.
- Remove `source[2]`, so that `source` becomes `"ab_aa"`.

Thus, the maximum number of removals is **1**.

Example 2

```
Input:
source = "bcda"
pattern = "d"
targetIndices = [0, 3]

Output: 2
```

Explanation:

We can remove both `source[0]` and `source[3]`, leaving `"cd"`. The pattern `"d"` remains a subsequence.

Thus, the maximum number of removals is **2**.

Example 3

```
Input:
source = "dda"
pattern = "dda"
targetIndices = [0, 1, 2]

Output: 0
```

Explanation:

We cannot remove any character because every character is needed to preserve `"dda"` as a subsequence. Thus, the maximum number of removals is **0**.

Example 4

```
Input:
source = "yeyeykyded"
pattern = "yeyyd"
targetIndices = [0, 2, 3, 4]

Output: 2
```

Explanation:

We can remove `source[2]` and `source[3]`, producing `"ye__ykyded"`. The pattern `"yeyyd"` remains a subsequence.

Thus, the maximum number of removals is **2**.

The required sequence `"aba"` is still a subsequence of `"abaa"`. Deleting index 1 is valid, but deleting two of the three eligible events would destroy every occurrence of the required sequence. The maximum number of deletions is therefore $\boxed{1}$.

Requirements

C1. DP formulation (5 points). Define a two-dimensional dynamic programming state using how much of `source` has been processed and how much of `pattern` has been matched. Specify the base cases, the recurrence, and the order in which the DP table is filled. Explain how your state handles an impossible partial match.

C2. Implementation (9 points). Implement a bottom-up dynamic programming algorithm that returns the *maximum number of deletions*. For each source character, consider whether it is used to match the next required character or left unmatched; an unmatched character can be deleted only if its index belongs to `targetIndices`. Do not use exhaustive enumeration of subsets as your main solution.

C3. Recover the removed indices (4 points). Return one optimal set of original indices to delete, in addition to the maximum count. Show that deleting exactly those indices leaves `pattern` as a subsequence. If no deletion is possible, return 0 and an empty list. Different optimal deletion sets are acceptable.

C4. Testing and analysis (2 points). Test cases with repeated characters, no allowable deletion, and more than one possible alignment. Give the time and auxiliary-space complexity in terms of $n = |\texttt{source}|$ and $m = |\texttt{pattern}|$.

Expected performance: $O(nm)$ time and $O(nm)$ auxiliary space for a 2D DP table and reconstruction.
Hint: The problem resembles aligning `pattern` with a subsequence of `source`; skipping an eligible

source character can earn one deletion.

Task D README, Documentation, and Testing (20 points)

As in Parts I and II, your programming portfolio must be reproducible: someone unfamiliar with your code should be able to run it, understand your algorithmic choices, and verify its output.

D1. README and running instructions (5 points). Include a `README.md` in the Part III project folder. State the purpose of each task, programming language and version, any required dependencies, how to compile or run each task (with actual commands), how to run the tests, and the input/output format. Provide at least one example command or invocation per task.

D2. Test cases and results (8 points). Provide the given example *plus at least three additional test cases for each of Tasks A–C* (at least four tests per task). Record input, expected output, and actual output for every test. You may provide runnable tests, a test script, or clearly documented manual tests. Include meaningful edge cases:

- **A:** empty delivery set; overlapping requests; requests that touch at an endpoint;
- **B:** zero budget; an item that exactly fits the budget; empty delivery set;
- **C:** zero possible removals; all eligible characters can be removed; repeated symbols with multiple possible alignments.

For Tasks A and B, check the optimum for *one small instance per task* independently (for example, by enumerating all subsets when there are at most 10 deliveries). For Task C, verify that removing your reported indices preserves the required subsequence.

D3. Algorithm documentation (5 points). Submit a short report (PDF) describing, for *each* of Tasks A–C, the DP state, base case, recurrence, the order in which the table is filled, and the running time and extra-space complexity. Explain how your implementation recovers the selected deliveries or removable log indices. The report should reference your test results and briefly note one lesson learned or challenge encountered.

D4. Submission and code organization (2 points). Submit one ZIP file through Canvas containing your source code, `README.md`, tests/test results, and report. Use descriptive filenames and meaningful comments. You may continue the repository from Parts I and II, but clearly identify the Part III files. If you used generative AI tools, acknowledge the tools and how they were used in your README, and independently verify the code and explanations.

Implementation policy: Use the required dynamic programming techniques for Tasks A–C. Standard-library sorting and basic data structures are allowed, but implement the core DP logic yourself. You may use a programming language of your choice.

Grading Summary

Component	Points
Task A: Maximum-Profit Delivery Scheduling	35
Task B: Delivery Selection with a Fuel Budget	25
Task C: Event Log Cleanup with Sequence Alignment	20
Task D: README, Documentation, and Testing	20
Total	**100**